ItsDangerous Cheat Sheet

Signed tokens for untrusted round trips · Python 3.10+ · zero deps

Signed ≠ encrypted. Anyone can base64-decode and read a token's payload. ItsDangerous only proves it wasn't modified.

PICK A CLASS

Sign raw bytes/str	`Signer`
+ timestamp / expiry	`TimestampSigner`
Sign objects (JSON)	`Serializer`
+ expiry	`TimedSerializer`
URL/cookie-safe, compressed	`URLSafeSerializer`
URL-safe + expiry (most common)	`URLSafeTimedSerializer`

CORE USAGE

```
from itsdangerous import (URLSafeTimedSerializer,
    BadSignature, SignatureExpired)
s = URLSafeTimedSerializer(SECRET_KEY, salt="pw-reset")
token = s.dumps({"user_id": 42})          # str
try:
    data = s.loads(token, max_age=3600)   # seconds
    data, ts = s.loads(token, max_age=3600,
                       return_timestamp=True)  # aware UTC
except SignatureExpired as e: ...         # e.date_signed
except BadSignature as e: ...             # e.payload
(unverified)
```

```
Signer(KEY).sign("v")            # b'v.SIG'
Signer(KEY).unsign(b"v.SIG")     # b'v' | BadSignature
Signer(KEY).validate(b"v.SIG")   # True / False
TimestampSigner(KEY).unsign(v, max_age=60,
                            return_timestamp=True)
```

CONSTRUCTOR KNOBS

Arg	Default	Meaning
`secret_key`	—	str/bytes or list oldest→newest; last signs, all verify
`salt`	`b"itsdangerous"` / Signer: `b"itsdangerous.Signer"`	unique per purpose
`serializer`	`json` (URL-safe: compact)	any `dumps`/`loads` obj
`signer_kwargs`	`{}`	sep, digest_method, key_derivation, algorithm
`fallback_signers`	`[]`	tried by `loads` only
`digest_method`	sha1 (in HMAC)	e.g. `hashlib.sha256`
`key_derivation`	`django-concat`	`concat`, `hmac`, `none`
`sep`	`b"."`	not in `A-Za-z0-9-_=`

SERIALIZER METHODS

`dumps(obj, salt=None)` · `loads(s, salt=None)` · `dump(obj, f)` · `load(f)` · `loads_unsafe(s)` → `(ok, payload)` *debug only* · `load_payload(b)` · `make_signer(salt)`

Timed: `loads(s, max_age=None, return_timestamp=False, salt=None)` — pass `salt=` by keyword.

EXCEPTIONS

```
BadData (.message)
├─ BadPayload (.original_error)       deserialize failed
└─ BadSignature (.payload)            tampered/key/salt
   ├─ BadHeader                       legacy
   └─ BadTimeSignature (.date_signed) bad timed sig
      └─ SignatureExpired             age > max_age / future
```

Catch `SignatureExpired` before `BadSignature`.

TOKEN FORMATS

`Signer`	`value.SIG`
`TimestampSigner`	`value.TS.SIG`
`URLSafe*`	`B64(json)[.TS].SIG`; leading `.` = zlib

RECIPES

```
# key rotation
URLSafeSerializer(["old-key", "new-key"])
# digest upgrade, old tokens still load
URLSafeSerializer(KEY,
  signer_kwargs={"digest_method": hashlib.sha256},
  fallback_signers=[{"digest_method": hashlib.sha1}])
# generate a key
python3 -c 'import os; print(os.urandom(16).hex())'
```

GOTCHAS

- No `max_age` → never expires. Tokens replay until expiry; enforce one-time use server-side.
- Use a different salt per purpose, or tokens work across contexts.
- Never use `pickle` as the serializer.
- `SignatureExpired` skips fallback signers.
- No `__version__`: `importlib.metadata.version("itsdangerous")`.

DEV COMMANDS

`uv sync` · `uv run pytest` · `uv run tox run -e typing` (mypy+pyright) · `-e style` (pre-commit/ruff) · `-e docs` · changelog → `CHANGES.rst` "Unreleased"